JobSheet - Week 6 – Java Stream API

Nama: Farrel Syauqy A

NIM: 251524069

Kelas: 1C

Repo GitHub: <Tuliskan Tautan Repo GitHub>

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W6_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W6_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Java Stream Sorting

Soal Praktikum
Instruksi Lengkapilah bagian kosong pada cuplikan kode di bawah ini dengan ketentuan sebagai berikut: 1. Bagian [1]: Panggil metode untuk mengubah ArrayList menjadi Stream. 2. Bagian [2]: Gunakan metode pengurutan (intermediate operation) yang menerima lambda expression. 3. Bagian [3]: Panggil metode getter yang sesuai untuk membandingkan nama antar objek. 4. Bagian [4]: Gunakan operasi terminal untuk merubah aliran data kembali menjadi objek List. 5. Bagian [5]: Gunakan utility class dan metode yang tepat untuk mengumpulkan hasil akhir ke dalam format List. **Source Code**

```
package JavaProblems;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ProblemOne {
   public static void main(String[] args) {
      ArrayList<Employee> list = new ArrayList<>();

      list.add(new Employee("John", 50000));
      list.add(new Employee("Rob", 70000));
      list.add(new Employee("Bob", 40000));
      list.add(new Employee("Alice", 10000));

      // Melakukan proses filtering/sorting menggunakan Stream API
      List<Employee> sortedEmp = list.[1]__________ // Ubah list ke stream
            // Mengurutkan employee berdasarkan nama
            .[2]__________((e1, e2) -> e1.[3]__________().compareTo(e2.getName()))
            // Mengumpulkan hasil akhir
            .[4]__________( [5]__________.toList() );

      for(Employee e : sortedEmp){
         System.out.println(e);
      }
   }
}

// Buatlah class Employee disertai dengan getter getName() dan getSalary(),
termasuk attribute dan method lainnya jika diperlukan!
```

Output yang Diharapkan

```
Employee{name='Alice', salary=50000}
Employee{name='Bob', salary=70000}
Employee{name='John', salary=10000}
Employee{name='Rob', salary=40000}
```

Source Code

File ProblemOne.java

```
package JavaProblem;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;



public class ProblemOne{
    public static void main (String[] args){
        ArrayList<Employee> list = new ArrayList<>();
```

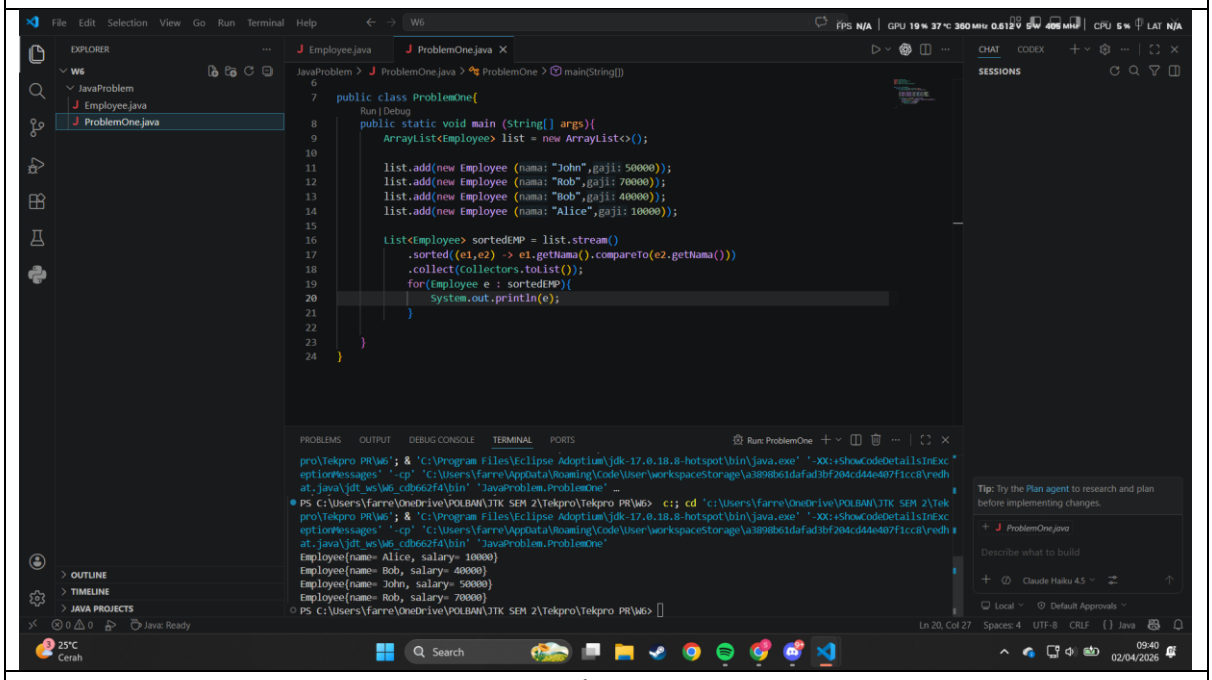
```
        list.add(new Employee ("John",50000));
        list.add(new Employee ("Rob",70000));
        list.add(new Employee ("Bob",40000));
        list.add(new Employee ("Alice",10000));

        List<Employee> sortedEMP = list.stream()
            .sorted((e1,e2) -> e1.getNama().compareTo(e2.getNama()))
            .collect(Collectors.toList());
            for(Employee e : sortedEMP){
                System.out.println(e);
            }

    }
}
```

FIle Employee.java

```
package JavaProblem;
public class Employee {
    private Integer gaji;
    private String nama;

    public Employee(String nama, Integer gaji){
        this.nama = nama;
        this.gaji = gaji;
    }

    /* Getter */
    public Integer getGaji(){
        return gaji;
    }

    public String getNama(){
        return nama;
    }

    /* Setter */
    public void setGaji(Integer gaji){
        this.gaji = gaji;
    }

    public void setNama(String nama){
        this.nama = nama;
    }
```

```
    public String toString(){
        String result = "Employee{name= " + getNama() +", salary= " +
getGaji() + "}";
        return result;


    }
}
```

Screenshot Hasil



Penjelasan

Stream pada java digunakan untuk mengolah sebuah kumpulan data secara deklaratif. **Stream** tidak menyimpan data, melainkan hanya mengolah data. **Stream** juga mendukung *functional programming* karena memanfaatkan pendekatan deklaratif (tidak menulis instruksi kode(imperatif)), memanfaatkan *High-Order functions* (fungsi yang menerima fungsi lain sebagai argumen), dan menjamin *Immutability* (ketidakubahan data) karena hanya mengolah data dan tidak menyimpannya, dan *Lazy Operation* dimana operasi akan dieksekusi ketika hasil pengolahannya diminta.

Proses menggunakan **stream** memiliki 3 tahapan utama:

1. **Source**
   Mengubah koleksi data menjadi **stream** dengan menggunakan *list.stream()*
2. **Intermediate Operation**
   Modul-modul yang bertugas mengolah data, tahap ini menghasilkan **stream baru** yang dapat disambung-sambung (chaining).
   Contohnya *filter(), map()* ,dan *sorted()*
3. **Terminal Operation**
   Modul Terakhir yang berguna untuk membungkus hasil dari operasi sebelumnya dan menutup **stream**. *Intermediate operation* hanya akan dijalankan ketika *Terminal operation* ada.

Pada Soal ini, List diubah menjadi sebuah **stream** dengan menggunakan *list.stream* untuk diolah datanya. Setelah diubah, maka selanjutnya tiap objek didalam list di bandingkan satu persatu menggunakan fungsi.*comparedTo()* jika objek pertama dibandingkan dengan objek kedua menghasilkan nilai < 0 maka objek pertama akan diletakan **didepan** objek kedua. Sebaliknya jika > 0, maka objek pertama akan diletakkan **dibelakang** objek kedua. Proses peletakan objek tersebut menggunakan fungsi *.sorted()*. Setelah semuanya terurut, objek-objek yang terurut tersebut disimpan ke dalam sebuah list menggunakan gabungan beberapa fungsi yaitu *collect(Collectors.toList())*.
Nama Teman Hal yang Dibantu (Opsional)
Farras: Membantu masalah terkait Package dan penamaan file dan publilc class yang tidak sesuai

No. 2 Multi-Level Sorting dengan Stream

Soal Praktikum

Skenario

Anda diminta untuk mengurutkan daftar objek Student. Aturan pengurutannya adalah:

1. Urutkan berdasarkan Nama secara alfabetis (tanpa mempedulikan huruf besar/kecil).
2. Jika ada nama yang sama, maka urutkan berdasarkan ID dari yang terkecil ke terbesar.

Instruksi Pengerjaan

Lengkapilah bagian kosong pada kode di bawah ini dengan ketentuan:

1. Bagian [1]: Panggil fungsi untuk memulai operasi Stream pada koleksi.
2. Bagian [2]: Gunakan metode Stream untuk melakukan pengurutan elemen.
3. Bagian [3]: Gunakan metode dari kelas wrapper Integer untuk membandingkan dua nilai ID secara aman.
4. Bagian [4]: Gunakan metode String yang tepat untuk membandingkan dua string tanpa memperhatikan case (huruf besar/kecil) untuk menghasilkan nilai integer perbandingan.
5. Bagian [5]: Panggil operasi terminal untuk mengumpulkan hasil akhir ke dalam sebuah List.

Instruksi

```
package JavaProblems;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class ProblemTwo {
   public static void main(String[] args) {

      List<Student> students = Arrays.asList(
            new Student(101, "Joe"),
            new Student(103, "Zulkifli"),
            new Student(102, "Riza"),
            new Student(104, "Alice"),
            new Student(105, "Joshua")
      );

      List<Student> ans = students.[1]__________ // Membuka stream
            .[2]__________((s1, s2) -> {      // Melakukan pengurutan

               // Logika: Jika nama sama, bandingkan ID
               if (s1.getName().equalsIgnoreCase(s2.getName())) {
                  return [3]__________.compare(s1.getId(), s2.getId());
```

```
            }
            // Jika nama berbeda, bandingkan Nama
            else {
              return s1.getName().[4]__________(s2.getName());
            }
          })
          .[5]__________(Collectors.toList()); // Mengumpulkan hasil

      for (Student student : ans) {
        System.out.println(student);
      }
    }
}

// Buatlah class Student disertai dengan attribute dan method lainnya yang
diperlukan!
```

Output yang Diharapkan

Student{id=104, name='Alice'}
Student{id=101, name='Joe'}
Student{id=105, name='Joshua'}
Student{id=102, name='Riza'}
Student{id=103, name='Zulkifli'}

Source Code

File ProlemTwo.java

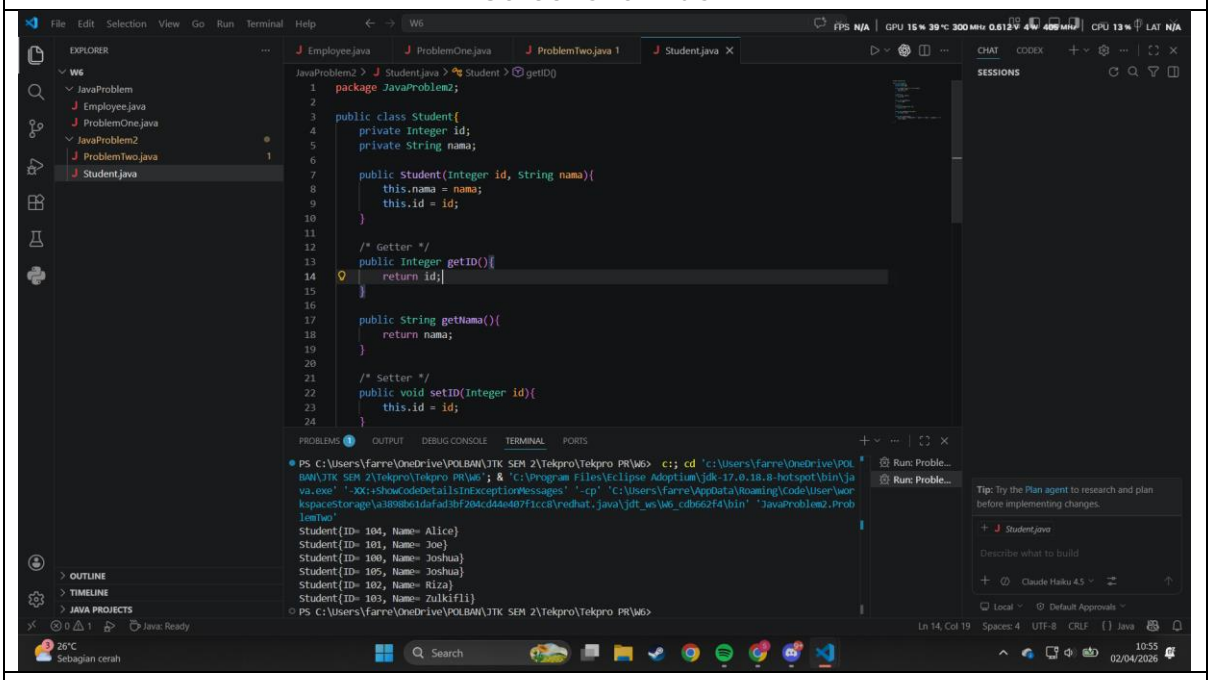
```java
package JavaProblem2;
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class ProblemTwo{
    public static void main (String[] args){
        List<Student> list = Arrays.asList(
            new Student(101, "Joe"),
            new Student(103, "Zulkifli"),
            new Student(102, "Riza"),
            new Student(104, "Alice"),
            new Student(105, "Joshua"),
            new Student(100,"Joshua")
        );
        List<Student> ans = list.stream()
            .sorted((s1,s2) ->{
                if (s1.getNama().equalsIgnoreCase(s2.getNama())){
                    return Integer.compare(s1.getID(),s2.getID());
                } else {
                    return s1.getNama().compareToIgnoreCase(s2.getNama());
                }
```

```
            })
            .collect(Collectors.toList());
        for(Student s : ans){
            System.out.println(s);
        }
    }
}
```

File Student

```
package JavaProblem2;

public class Student{
    private Integer id;
    private String nama;

    public Student(Integer id, String nama){
        this.nama = nama;
        this.id = id;
    }

    /* Getter */
    public Integer getID(){
        return id;
    }

    public String getNama(){
        return nama;
    }

    /* Setter */
    public void setID(Integer id){
        this.id = id;
    }

    public void setNama(String nama){
        this.nama = nama;
    }

    public String toString(){
        String result = "Student{ID= " + getID() +", Name= " + getNama() +
"}";
        return result;

    }
}
```

Screenshot Hasil



Penjelasan

Arrays.asList adalah metode praktis yang berada pada *java.util.Arrays,* berfungsi sebagai untuk mengubah kumpulan data menjadi sebuah list. **Arrays.asList** memiliki 3 sifat utama, yaitu :

1. List ciptaan dari Metode ini menghasilkan list yang ukurannya tetap, artinya fungsi *add()* dan *remove()* tidak bekerja dan akan menyebabkan error.
2. **Arrays.asList** sebenarnya tidak menciptakan struktur nya dari nol, ia bertindak hanya **wrapper** dari **array aslinya.** Jika fungsi *set()* digunakan pada list tersebut untuk mengubah nilai, maka nilai di array aslinya berubah juga. Begitupun sebaliknya.
3. Meskipun tipe datanya ditulis sebagai List, objek asli di belakang layar yang dihasilkan bukanlah *java.util.ArrayList* yang dinamis, melainkan kelas internal rahasia bernama *java.util.Arrays$ArrayList*

equalsIgnoreCase() digunakan pada kode ini untuk mengecek apakah kedua buah objek String sama atau tidak tanpa memperdulikan huruf besar atau huruf kecil. Tipe pengembalian fungsi ini adalah **boolean.**

compareToIgnoreCase() digunakan pada kode ini untuk membandingkan kedua buah objek String tanpa memperdulikan Huruf besar atau huruf kecil. Tipe return dari fungsi ini adalah **Integer** dimana jika hasilnya < 0 maka teks pertama urutan abjadnya **lebih kecil** dari teks kedua, jika hasilnya > 0 maka teks pertama urutan abjadnya **lebih besar** dari teks kedua, dan jika hasilnya 0 maka kedua teks memiliki urutan abjad yang sama.

Integer.compare() digunakan pada kode ini untuk membandingkan kedua buah objek integer. Tipe return fungsi ini adalah **Integer** dimana jika hasilnya <0 maka angka pertama **lebih kecil** dari angka kedua, jika hasilnya >0 maka angka pertama **lebih besar** dari angka kedua, dan jika hasilnya 0 maka kedua angka sama besarnya.

Nama Teman Hal yang Dibantu (Opsional)

No. 3 Finding the Nth Largest Element

Soal Praktikum

Skenario

Anda memiliki sebuah array integer primitif. Anda diminta untuk mencari elemen terbesar ke-3 dari array tersebut menggunakan Java 8 Stream. Tahapannya adalah: mengubah array menjadi stream, mengubah tipe data primitif menjadi objek agar bisa diurutkan secara kustom, mengurutkan dari yang terbesar, melompati elemen tertentu, dan mengambil hasilnya.

Instruksi

Isilah bagian rumpang pada kode di bawah ini dengan ketentuan:

1. Bagian [1]: Panggil metode dari kelas Arrays untuk mengubah array menjadi IntStream.
2. Bagian [2]: Gunakan metode untuk mengubah stream primitif (int) menjadi stream objek (Integer) agar bisa menggunakan Comparator.
3. Bagian [3]: Gunakan operasi untuk melompati (skip) sejumlah elemen pertama.
4. Bagian [4]: Gunakan operasi terminal untuk mengambil elemen pertama yang tersedia setelah proses skipping.
5. Bagian [5]: Gunakan metode untuk memberikan nilai default jika stream ternyata kosong.

```
package JavaProblems;

import java.util.Arrays;

public class ProblemThree {
   public static void main(String[] args) {
      int arr[] = {12, 4, 3, 1, 9, 657};
      int n = 3; // Mencari elemen terbesar ke-3

      int ans = [1]__________.stream(arr)
            .[2]__________()                    // Konversi ke Stream<Integer>
            .sorted((a, b) -> Integer.compare(b, a))    // Urutkan descending
            .[3]__________(n - 1)                // Lewati 2 elemen pertama
            .[4]__________()                    // Ambil elemen yang tersisa
            .[5]__________(0);                  // Nilai default jika tidak ada

      System.out.println("Elemen terbesar ke-3 adalah: " + ans);
   }
}
```

Output yang Diharapkan

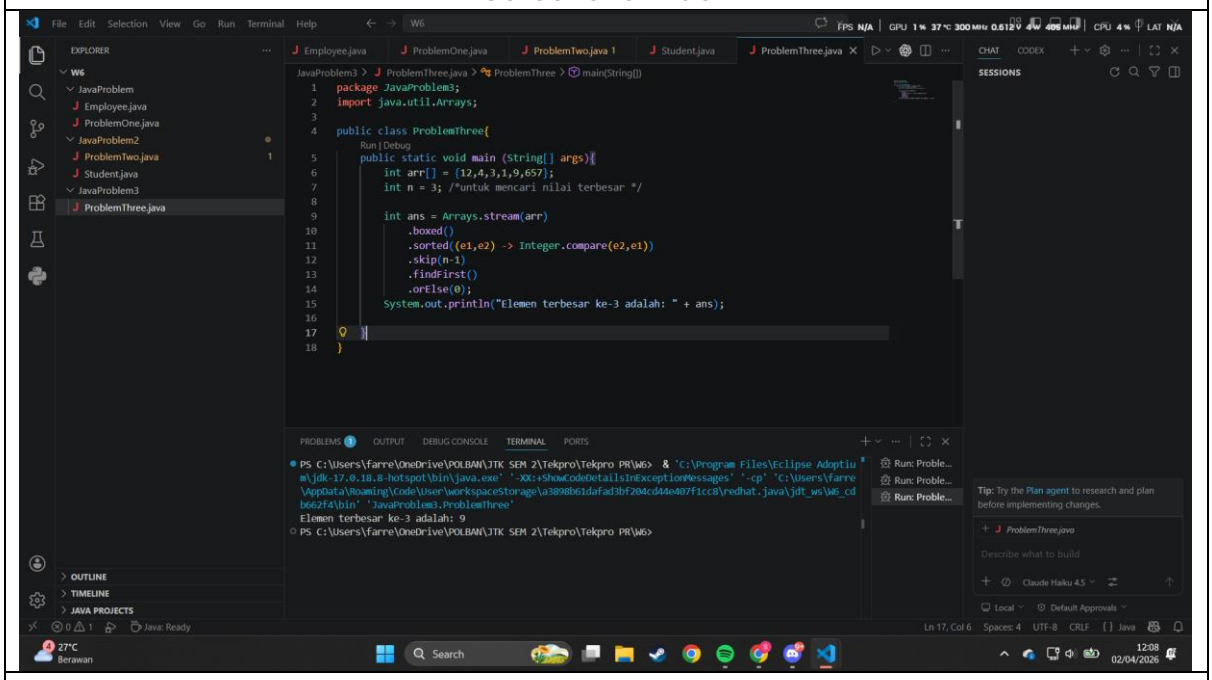
The 3rd largest element is: 9

Source Code

File ProblemThree.java

```java
package JavaProblem3;
import java.util.Arrays;

public class ProblemThree{
    public static void main (String[] args){
        int arr[] = {12,4,3,1,9,657};
        int n = 3; /*untuk mencari nilai terbesar */

        int ans = Arrays.stream(arr)
            .boxed()
            .sorted((e1,e2) -> Integer.compare(e2,e1))
            .skip(n-1)
            .findFirst()
            .orElse(0);
        System.out.println("Elemen terbesar ke-3 adalah: " + ans);

    }
}
```

Screenshot Hasil



Penjelasan

Arrays.stream() adalah sebuah fungsi yang merubah **array biasa** menjadi **stream.** **Arrays.stream()** dapat mengubah **array objek** menjadi **stream<objek>** biasa. Sedangkan **array primitif** diubah menjadi versi stream yg lebih efisien yaitu **primitive stream,** contohnya jika int arr[] diubah menjadi stream dengan **Arrays.stream(arr)** maka akan berubah menjadi **InStream.** Lehih efisien karena data yang mengalir adalah data **primitif** bukan **objek.**

Arrays.stream() juga dapat menentukan data mana aja yang mau diambil. Misal

pada int arr[] ingin mengambil data dari index 1-4 saja, maka cukup ketikan fungsi **Arrays.stream(arr,1,4).** Pada kode ini, **Array.stream()** berguna untuk mengubah **array primitf arr** menjadi **stream primitif**, kemudian **stream primitif** diubah menjadi **stream objek** dengan menggunakan **boxed().** Setiap objek kemudian dibandingkan dan diurutkan **descending** menggunakan gabungan fungsi **sorted()** dan **compareTo().** Setelah diurutkan, **skip()** digunakan untuk melewati 2 objek pertama. Untuk mengambil elemen pertama setelah melewati 2 objek pertama, **findFirst()** digunakan untuk mengambil objek pertama setelah 2 objek dilewati. Apabila ternyata **stream kosong,** maka nilai 0 akan dikembalikan dengan menggunakan fungsi **orElse().**
Nama Teman Hal yang Dibantu (Opsional)